Unitary layer simulation phase error

https://github.com/tensorflow/quantum/issues/326

ROW 80

Unitary layer simulation phase error #326

New issue

Closed

we-taper opened on Aug 3, 2020 · edited by we-taper

Edits

It is strange to realize that `S[2] CX[1,2] S[2]` is not equivalent to `CY` in TFQ simulation.

Also, `S X S^dag != Y` in TFQ simulation. Although their difference in TFQ is a phase, maybe that is the cause for the problem above?

Example:

```
import tensorflow_quantum as tfq
import cirq
import numpy as np

np.set_printoptions(precision=3, suppress=True)
y = np.array([[0, -1j], [1j, 0]])
cy = np.array([[1, 0, 0, 0], [0, 1, 0, 0], [0, 0, 0, -1j], [0, 0, 1j, 0]])


def get_unitary(inc) -> np.ndarray:
    return tfq.layers.Unitary()(inc).to_tensor().numpy().squeeze()


q = cirq.GridQubit.rect(1, 2)

circuit = cirq.Circuit(cirq.CNOT.on(q[0], q[1]))
cx = get_unitary(circuit)
print(cx)
# [[1.+0.j 0.+0.j 0.+0.j 0.+0.j]
#  [0.+0.j 1.+0.j 0.+0.j 0.+0.j]
#  [0.+0.j 0.+0.j 0.-0.j 1.+0.j]
#  [0.+0.j 0.+0.j 1.+0.j 0.-0.j]]

circuit = cirq.Circuit(
    cirq.S.on(q[1]), cirq.CNOT.on(q[0], q[1]), (cirq.S ** -1).on(q[1])
)
s_cnot_sdag = get_unitary(circuit)
print(s_cnot_sdag)
# [[ 1.+0.j  0.+0.j  0.+0.j  0.+0.j]
#  [ 0.+0.j  1.+0.j  0.+0.j  0.+0.j]
#  [ 0.+0.j  0.+0.j  0.-0.j -0.+1.j]
#  [ 0.+0.j  0.+0.j  0.-1.j  0.-0.j]]
print(s_cnot_sdag - cy)  # should be 0!
# [[ 0.+0.j  0.+0.j  0.+0.j  0.+0.j]
#  [ 0.+0.j  0.+0.j  0.+0.j  0.+0.j]
#  [ 0.+0.j  0.+0.j  0.-0.j -0.+2.j]
#  [ 0.+0.j  0.+0.j  0.-2.j  0.-0.j]]


circuit = cirq.Circuit(cirq.S.on(q[0]), cirq.X.on(q[0]), (cirq.S ** -1).on(q[0]),)
sxsdag = get_unitary(circuit)
print(sxsdag)
# [[ 0.-0.j -0.+1.j]
#  [ 0.-1.j  0.-0.j]]
circuit = cirq.Circuit(cirq.S.on(q[0]))
s = get_unitary(circuit)
print(s)
# [[ 1.+0.j  0.+0.j]
#  [ 0.+0.j -0.+1.j]]
circuit = cirq.Circuit(cirq.X.on(q[0]))
x = get_unitary(circuit)
print(x)
# [[0.-0.j 1.+0.j]
#  [1.+0.j 0.-0.j]]
circuit = cirq.Circuit((cirq.S ** -1).on(q[0]))
sdag = get_unitary(circuit)
print(sdag)
# [[ 1.+0.j  0.+0.j]
#  [ 0.+0.j -0.-1.j]]
print(np.allclose(s @ x @ sdag, y, atol=1e-5))
# True
```

Assignees

MichaelBroughton

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

```
print(np.allclose(s @ x @ sdag, y, atol=1e-5))
# True
print(np.allclose(sxsdag, y, atol=1e-5))
# False !! This should be true
print(s @ x @ sdag - sxsdag)
# [[ 0.+0.j  0.-2.j]
#  [-0.+2.j  0.+0.j]]
```

we-taper changed the title ~~Unitary gate error~~ Unitary layer simulation phase error on Aug 3, 2020

MichaelBroughton on Aug 3, 2020 — Collaborator

Would you mind rerunning this snippet and changing `get_unitary` to be:

```
def get_unitary(inc) -> np.ndarray:
    return cirq.unitary(inc)
```

And see if any of the outputs change ? This would indicate that TFQ does indeed behave differently from Cirq (which would be a big issue). I also have this vague memory of some Cirq gates having some interesting values of "default global phase" when working on parts of TFQ, which could maybe be the root cause here too ?

MichaelBroughton self-assigned this on Aug 3, 2020

we-taper on Aug 3, 2020 · edited by we-taper — Edits — Contributor — Author

Sorry, I realize this silly mistake in my script: the quantum gates execute in the reverse direction of matrix multiplication. The correction script (below) clearly shows that Cirq/TFQ simulations are correction.

```
import tensorflow_quantum as tfq
import cirq
import numpy as np

y = np.array([[0, -1j], [1j, 0]])
cy = np.array([[1, 0, 0, 0], [0, 1, 0, 0], [0, 0, 0, -1j], [0, 0, 1j, 0]])


def get_unitary(inc) -> np.ndarray:
    return tfq.layers.Unitary()(inc).to_tensor().numpy().squeeze()


q = cirq.GridQubit.rect(1, 2)

circuit = cirq.Circuit(
    (cirq.S ** -1).on(q[1]), cirq.CNOT.on(q[0], q[1]), cirq.S.on(q[1])
)
s_cnot_sdag = get_unitary(circuit)
print(np.allclose(s_cnot_sdag, cy, atol=1e-5))
# True
circuit = cirq.Circuit((cirq.S ** -1).on(q[0]), cirq.X.on(q[0]), cirq.S.on(q[0]),)
sxsdag = get_unitary(circuit)
print(np.allclose(sxsdag, y, atol=1e-5))
# True


circuit = cirq.Circuit(
    cirq.rz(-0.5 * np.pi).on(q[1]),
    cirq.CNOT.on(q[0], q[1]),
    cirq.rz(0.5 * np.pi).on(q[1]),
)
rz_cnot_rzdag = get_unitary(circuit)
print(np.allclose(rz_cnot_rzdag, cy, atol=1e-5))
# True
```